

Automated Micro-Typography and Dynamic Engine Selection for Modern L^AT_EX Publishing Workflows

Dr. Alexander V. Mercer, Dr. Elena Rostova,
Marcus Vance

Abstract

Modern automated publishing platforms require robust document processing pipelines that deliver high-quality typographic output while accommodating diverse input documents. This paper presents an integrated framework developed jointly by Vertex Institute and Synapse Systems for automated micro-typographic refinement, engine feature detection, and dynamic optimization of line-breaking parameters in L^AT_EX. By coupling character protrusion and font expansion with adaptive paragraph penalty scoring, our approach eliminates overfull boxes and minimizes unwanted hyphenation across multi-column journal layouts. Benchmark evaluations performed on the Meridian Document Corpus demonstrate a 42% reduction in typographic badness metrics and a 35% improvement in visual balance compared to unoptimized compilation pipelines.

1 Introduction

The digital publishing ecosystem relies heavily on document compilation engines to transform structured content into fixed-layout output such as PDF. Within the T_EX community, pdfT_EX, X_YT_EX, and LuaT_EX represent the primary engines, each offering distinct capabilities regarding font handling, Unicode support, and micro-typographic adjustments. However, managing multi-engine document processing pipelines across heterogeneous publishing repositories remains a challenging software engineering problem.

Traditional L^AT_EX compilation workflows often apply static formatting rules that fail to adapt to local page geometry constraints. In multi-column publishing contexts, such as *TUGboat* articles, rigid line-breaking settings frequently trigger overfull `\hbox` warnings or create uneven vertical spacing. To address these issues, we introduce the AMTS (*Automated Micro-Typography System*), an open-framework architecture designed to optimize micro-spacing and engine parameters dynamically during document compilation.

2 Architecture of the Automated Pipeline

The AMTS framework consists of three principal modules operating sequentially during compilation:

1. **Engine Feature Detector:** Evaluates the active compiler environment (pdfT_EX, LuaT_EX, or

X_YT_EX) and enables engine-specific primitives for character protrusion and font expansion.

2. **Adaptive Parameter Tuner:** Adjusts line-breaking parameters such as `\pretolerance`, `\tolerance`, and `\emergencystretch` based on column width and font metrics.
3. **Quality Auditor:** Scans the compilation log for badness scores, overfull boxes, and orphan lines, performing multi-pass tuning when threshold bounds are exceeded.

2.1 Engine Feature Detection

Engine detection is performed during the early execution phase using conditional T_EX primitives. When compiling under pdfT_EX or LuaT_EX, typographic features such as margin protrusion and font expansion are activated dynamically. Under X_YT_EX, font features are managed through the `fontspec` interface while maintaining backward compatibility with standard font declarations.

2.2 Micro-Typographic Refinements

Micro-typography enhances readability by making subtle adjustments to character margins and font widths. Character protrusion (hanging punctuation) allows small punctuation marks such as periods, commas, and hyphens to extend slightly into the right margin, creating a visually straight edge. Font expansion slightly stretches or shrinks glyphs horizontally to avoid awkward line breaks without introducing noticeable distortion.

When compiled under pdfT_EX or LuaT_EX, the AMTS pipeline configures parameters with controlled expansion limits:

```
\usepackage[
  activate={true,nocompatibility},
  final, tracking=true,
  kerning=true, spacing=true,
  factor=1100, stretch=10,
  shrink=10
]{microtype}
```

These parameters ensure that font expansion remains within a $\pm 1\%$ envelope, preserving the structural integrity of the typeface while providing sufficient flexibility for line-breaking algorithms.

3 Mathematical Formalization

To evaluate paragraph quality objectively, we define a composite penalty score P for each paragraph containing n lines. The total penalty combines line badness b_i , hyphenation occurrence h_i , and margin protrusion variance d_i :